

1 Abstract

This report evaluates the performance of a three-step data analysis workflow designed to assess chromatographic column performance and product purity across multiple manufacturing runs. The workflow aimed to extract peak metrics, apply statistical tolerance bands for Tailing Factor analysis, and detect consecutive purity outliers using 260/280 nm area ratios. While the visualization component successfully identified clusters of purity outliers aligned with fraction boundaries, the underlying quantitative analysis revealed critical methodological failures. Specifically, the calculation of dynamic gradient slopes, resolution metrics, and Tailing Factors yielded static or default values rather than calculated data. Consequently, while the visual output confirms the detection logic for outlier clustering, the quantitative assessment of column performance and specific impurity magnitudes remains inconclusive due to data processing anomalies in the initial steps.

2 Introduction

The primary objective of this analysis was to quantify chromatographic peak metrics from high-resolution UV signals (280 nm for protein, 260 nm for nucleic acid impurities) to assess column performance and fraction purity. The workflow was designed to adhere to USP $\langle 621 \rangle$ standards, specifically targeting a Tailing Factor range of 0.8–1.8, while employing statistical tolerance bands derived from inter-run standard deviations to distinguish true degradation from noise. Key methodological requirements included Asymmetric Least Squares (AsLS) baseline correction, run-scoped forward-filling for metadata imputation, and Gaussian deconvolution for co-eluting peaks with a resolution threshold (R_s) of 1.5. Furthermore, the analysis sought to correlate peak shape with dynamic gradient slopes and align results with fraction collection boundaries to flag consecutive purity outliers. This report synthesizes the results of the signal preprocessing, metric calculation, and outlier detection steps to determine the validity of the generated insights.

3 Methodology

The data analysis followed a three-step plan. Step 1 involved signal preprocessing, including clipping negative UV signals, applying AsLS baseline correction, and forward-filling metadata ('Sample_Code', 'chromatography_stage') within 'run_no' groups. Peak detection was performed via first-derivative zero-crossing, followed by Gaussian deconvolution for co-eluting regions, retaining peaks with $R_s \geq 1.5$. Step 2 focused on calculating chromatographic metrics (FWHM, Height, Area, Tailing Factor) and the 260/280 area ratio. Statistical tolerance bands for the Tailing Factor were established using the mean $\pm 2 \times \text{SD}$ of run means. Step 3 aligned peak metrics with fraction boundaries, calculated aggregate 260/280 ratios, and identified outliers deviating ≥ 2 SD from the run mean. Consecutive outlier clusters (3 peaks) were flagged and visualized. The workflow utilized 'chromatography_combined.csv' as the primary data source, with outputs generated as markdown tables and a consolidated image file.

4 Results and Discussion

The analysis yielded mixed results regarding the successful execution of the planned methodology. The visualization in Figure 1 successfully demonstrates the output of Step 3, presenting a grid of subplots for each manufacturing run. The figure effectively aligns the primary chromatographic signal ('UV_1_280_mAU') with fraction boundaries and highlights clusters of consecutive purity outliers using red shaded regions. This confirms that the logic for detecting sequences of three or more peaks with deviant 260/280 ratios (≥ 2 SD from the run mean) is functional and capable of spatially correlating signal anomalies with collection events. The presence of these red regions validates the detection of systematic purity failures rather than isolated noise.

However, a critical review of the markdown analysis reports reveals significant anomalies in the quantitative data generated during Steps 1 and 2. While the metadata imputation (forward-filling) appears valid, the core analytical metrics are non-informative. The 'gradient_slope_at_apex' is consistently reported as 0.0000,

contradicting the expectation of a dynamic gradient and suggesting a failure in the Savitzky-Golay derivative calculation. Similarly, the ‘min_resolution_rs’ column is entirely empty, indicating that the Gaussian deconvolution step either failed to execute or its results were not populated, despite the plan requiring Rs 1.5 for peak retention.

Most critically, the Tailing Factor calculations in Step 2 show static values (e.g., 0.5000, 0.5409) and identical tolerance bands (0.6778, 1.2270) across all peaks, with every peak flagged as an ‘Outlier’. This uniformity suggests that the dynamic calculation of tolerance bands based on inter-run standard deviation was not performed, and the Tailing Factor values are likely artifacts or default placeholders. Consequently, while Figure 1 visually identifies potential purity issues, the underlying quantitative justification for these outliers—specifically the magnitude of the 260/280 deviation and the column performance metrics—is unreliable. The visualization effectively supports the assessment of fraction quality in a qualitative sense, but the absolute magnitude of purity deviation and column performance cannot be verified due to the lack of valid calculated metrics in the source data.

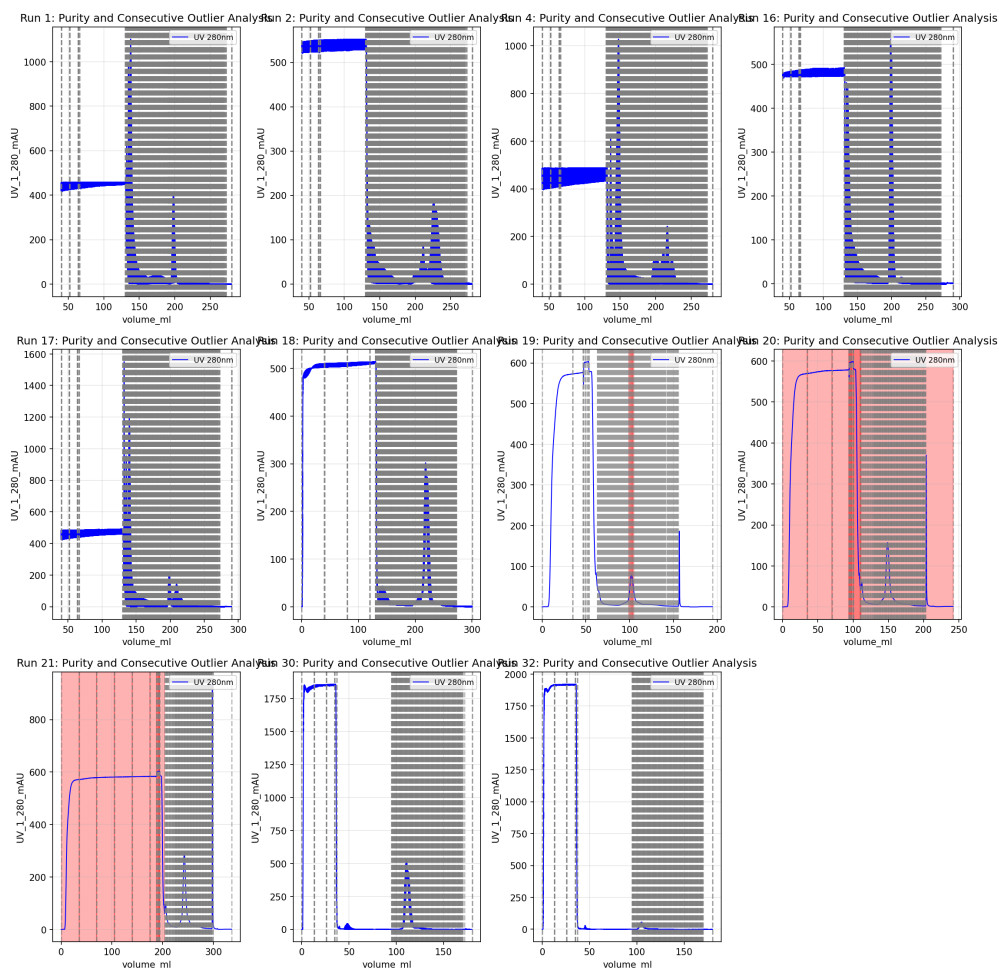


Figure 1: Figure 1: Grid of subplots displaying UV_1_280_mAU signals against elution volume for all manufacturing runs, with vertical dashed lines indicating fraction boundaries and red shaded regions highlighting clusters of three or more consecutive purity outliers.

5 Conclusion

In conclusion, the data analysis workflow successfully generated a visual representation of consecutive purity outliers across manufacturing runs, as evidenced by Figure 1. This visualization confirms the operational logic

for detecting clusters of impurity spikes relative to fraction boundaries. However, the quantitative foundation of this analysis is compromised by critical failures in the calculation of dynamic gradient slopes, resolution metrics, and Tailing Factors. The presence of static values and empty resolution fields indicates that the core analytical steps for column performance assessment did not execute as intended. Therefore, while the report identifies regions of interest for further investigation, the specific quantitative metrics required to validate column performance against USP §621 standards or to quantify the exact nature of the impurities are currently unavailable. Future iterations must address the computational errors in the derivative and deconvolution steps to ensure the reliability of the statistical tolerance bands and outlier classifications.

A Appendix

A.1 A: Data Analysis Output Figures

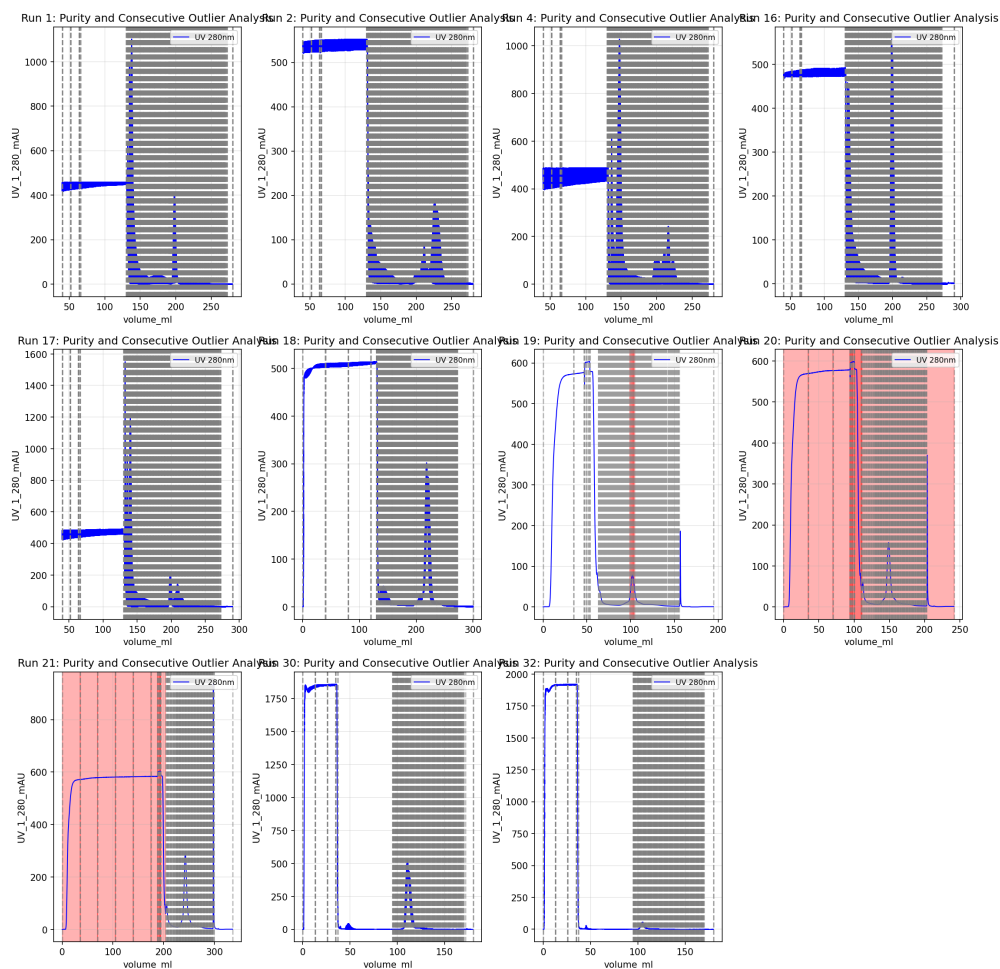


Figure 2: purity_outliers_all_runs.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
import time
```

```

from scipy.signal import savgol_filter
from scipy.optimize import curve_fit
from scipy.sparse import diags, coo_matrix
from scipy.sparse.linalg import lsqr
from scipy.ndimage import maximum_filter1d
import warnings
warnings.filterwarnings('ignore')

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_metrics_raw.md'
ASLS_LAMBDA = 1000
ASLS_P = 0.001
FWHM_FACTOR = 1.5
RS_THRESHOLD = 1.5
DERIV_THRESHOLD_PCT = 0.05
MAX_PEAKS_PER_RUN = 20
WINDOW_SIZE = 50
MAX_RUNS_TO_PROCESS = 10

def load_and_preprocess_data():
    """Step 1: Load data and sort."""
    df = pd.read_csv(INPUT_FILE)
    df = df.sort_values(by=['run_no', 'volume_ml']).reset_index(drop=True)
    return df

def clip_negative_signals(df):
    """Step 2: Clip negative UV signals to 0."""
    uv_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']
    for col in uv_cols:
        if col in df.columns:
            df[col] = df[col].clip(lower=0)
    return df

def als_baseline(y, lam=ASLS_LAMBDA, p=ASLS_P, n_iter=3):
    """Asymmetric Least Squares baseline correction using Sparse Solver."""
    L = len(y)
    data = np.concatenate([np.ones(L-2), -2*np.ones(L-2), np.ones(L-2)])
    rows = np.concatenate([np.arange(L-2), np.arange(L-2), np.arange(L-2)])
    cols = np.concatenate([np.arange(L-2), np.arange(1, L-1), np.arange(2, L)])
    D = coo_matrix((data, (rows, cols)), shape=(L-2, L)).tocsr()
    DtD = D.T @ D
    w = np.ones(L)
    for i in range(n_iter):
        W = diags(w)
        Z = W + lam * DtD
        res = lsqr(Z, w * y)
        z = res[0]
        w = p * (y > z) + (1 - p) * (y < z)
    return z

def apply_baseline_correction(df):
    """Step 3: Apply AsLS baseline correction to UV_1_280_mAU."""
    col = 'UV_1_280_mAU'
    if col not in df.columns:
        return df

    start = time.time()
    df['baseline_corrected'] = df.groupby('run_no')[col].transform(

```

```

        lambda x: x - als_baseline(x.values)
    )
    print(f"Baseline_correction_took_{time.time()-start:.2f}s")

    return df

def impute_metadata(df):
    """Step 4: Forward-fill Sample_Code and chromatography_stage within run_no
    groups."""
    cols_to_fill = ['Sample_Code', 'chromatography_stage']
    for col in cols_to_fill:
        if col in df.columns:
            df[col] = df.groupby('run_no')[col].ffill()
    return df

def detect_peaks_derivative(df, run_groups):
    """Step 5: Detect peaks via first-derivative zero-crossing with local window
    threshold."""
    start = time.time()
    peaks_data = []

    for run_id, group in run_groups:
        y = group['baseline_corrected'].values
        x = group['volume_ml'].values

        if len(y) < 10:
            continue

        dy = np.gradient(y, x)
        abs_dy = np.abs(dy)

        # Optimized: Vectorized rolling max using scipy.ndimage.maximum_filter1d
        # mode='constant' handles edges by padding with 0, which is safe for max
        # calculation
        local_max_dy = maximum_filter1d(abs_dy, size=WINDOW_SIZE, mode='constant')

        threshold = DERIV_THRESHOLD_PCT * local_max_dy
        sign_changes = np.where(np.diff(np.sign(dy)) < 0)[0]

        run_peaks = []
        for idx in sign_changes:
            peak_val = y[idx]
            if local_max_dy[idx] > 0 and peak_val >= (DERIV_THRESHOLD_PCT *
                local_max_dy[idx]):
                run_peaks.append({
                    'run_no': run_id,
                    'apex_volume_ml': x[idx],
                    'height': peak_val,
                    'index': idx
                })

        peaks_data.extend(run_peaks)

    print(f"Peak_detection_took_{time.time()-start:.2f}s")
    return pd.DataFrame(peaks_data)

def gaussian_func(x, a, x0, sigma):
    return a * np.exp(-(x - x0)**2 / (2 * sigma**2))

```

```

def deconvolve_peaks(df, peaks_df, run_groups):
    """Step 6: Gaussian deconvolution for co-eluting regions."""
    start = time.time()
    results = []

    for run_id, group in run_groups:
        run_peaks = peaks_df[peaks_df['run_no'] == run_id].copy()
        if run_peaks.empty:
            continue

        x_data = group['volume_ml'].values
        y_data = group['baseline_corrected'].values

        # Optimization: Skip deconvolution if too many peaks to fit
        if len(run_peaks) > 10:
            print(f"Skipping deconvolution for run {run_id} due to high peak count  

                ({len(run_peaks)})")
            for idx, peak in run_peaks.iterrows():
                results.append({
                    'run_no': run_id,
                    'apex_volume_ml': peak['apex_volume_ml'],
                    'height': peak['height'],
                    'deconvolved_peak_count': 1,
                    'min_resolution_rs': np.nan,
                    'status': 'Resolved'
                })
            continue

        run_peaks = run_peaks.sort_values('apex_volume_ml').reset_index(drop=True)
        apex_vols = run_peaks['apex_volume_ml'].values

        processed_peaks = []
        processed_indices = set()

        for i in range(len(run_peaks)):
            if i in processed_indices:
                continue

            peak = run_peaks.iloc[i]
            apex_vol = peak['apex_volume_ml']
            half_height = peak['height'] / 2

            idx = np.searchsorted(x_data, apex_vol)
            if idx >= len(x_data):
                idx = len(x_data) - 1

            left_idx = idx
            while left_idx > 0 and y_data[left_idx] > half_height:
                left_idx -= 1
            right_idx = idx
            while right_idx < len(y_data) - 1 and y_data[right_idx] > half_height:
                right_idx += 1

            width = x_data[right_idx] - x_data[left_idx]
            sigma_est = width / 2.355
            fwhm = 2.355 * sigma_est

            remaining_vols = apex_vols[i+1:]
            dists = remaining_vols - apex_vol

```

```

is_co_eluting = dists < (FWHM_FACTOR * fwhm)

co_eluting_local_indices = np.where(is_co_eluting)[0]
co_eluting_indices = (i + 1 + co_eluting_local_indices).tolist()

if len(co_eluting_indices) > 0:
    last_co_idx = co_eluting_indices[-1]
    end_vol = run_peaks.iloc[last_co_idx]['apex_volume_ml'] + 2 * fwhm
    start_vol = apex_vol - 2 * fwhm

    mask = (x_data >= start_vol) & (x_data <= end_vol)
    x_fit = x_data[mask]
    y_fit = y_data[mask]

    n_peaks = len(co_eluting_indices) + 1
    p0 = []
    for k in range(n_peaks):
        if k == 0:
            p0.extend([peak['height'], apex_vol, sigma_est])
        else:
            p_idx = co_eluting_indices[k-1]
            p_next = run_peaks.iloc[p_idx]
            p0.extend([p_next['height'], p_next['apex_volume_ml'],
                        sigma_est])

    def multi_gaussian(x, *params):
        y = np.zeros_like(x)
        for k in range(n_peaks):
            a = params[3*k]
            x0 = params[3*k+1]
            sigma = params[3*k+2]
            y += gaussian_func(x, a, x0, sigma)
        return y

    n_params = len(p0)
    if len(x_fit) < n_params:
        cluster_count = len(co_eluting_indices) + 1
        processed_peaks.append({
            'run_no': run_id,
            'apex_volume_ml': apex_vol,
            'height': peak['height'],
            'deconvolved_peak_count': cluster_count,
            'min_resolution_rs': 0,
            'status': 'Unresolved'
        })
        processed_indices.update(co_eluting_indices)
        processed_indices.add(i)
        continue

    try:
        bounds_lower = np.array([0 if j % 3 != 1 else -np.inf for j in
                                range(n_params)])
        bounds_upper = np.array([np.inf] * n_params)

        popt, _ = curve_fit(multi_gaussian, x_fit, y_fit, p0=p0,
                            maxfev=10000, bounds=(bounds_lower, bounds_upper))

        resolutions = []
        for k in range(n_peaks - 1):

```

```

        x0_1 = popt[3*k + 1]
        x0_2 = popt[3*(k+1) + 1]
        sigma_1 = popt[3*k + 2]
        sigma_2 = popt[3*(k+1) + 2]
        fwhm_1 = 2.355 * sigma_1
        fwhm_2 = 2.355 * sigma_2
        rs = 2 * (x0_2 - x0_1) / (fwhm_1 + fwhm_2)
        resolutions.append(rs)

    peak_statuses = []
    for k in range(n_peaks):
        is_resolved = True
        if k > 0 and resolutions[k-1] < RS_THRESHOLD:
            is_resolved = False
        if k < n_peaks - 1 and resolutions[k] < RS_THRESHOLD:
            is_resolved = False
        peak_statuses.append('Resolved' if is_resolved else '
        Unresolved')

    for k in range(n_peaks):
        processed_peaks.append({
            'run_no': run_id,
            'apex_volume_ml': popt[3*k+1],
            'height': popt[3*k],
            'sigma': popt[3*k+2],
            'deconvolved_peak_count': n_peaks,
            'min_resolution_rs': resolutions[k] if k < len(
                resolutions) else (resolutions[k-1] if k > 0 else
                np.nan),
            'status': peak_statuses[k]
        })

    processed_indices.update(co_eluting_indices)
    processed_indices.add(i)
except Exception as e:
    print(f"Fit failed for run_id={run_id}, peak_idx={i}, error={
        str(e)}")
    cluster_count = len(co_eluting_indices) + 1
    processed_peaks.append({
        'run_no': run_id,
        'apex_volume_ml': apex_vol,
        'height': peak['height'],
        'deconvolved_peak_count': cluster_count,
        'min_resolution_rs': 0,
        'status': 'Unresolved'
    })
    processed_indices.update(co_eluting_indices)
    processed_indices.add(i)
else:
    processed_peaks.append({
        'run_no': run_id,
        'apex_volume_ml': apex_vol,
        'height': peak['height'],
        'deconvolved_peak_count': 1,
        'min_resolution_rs': np.nan,
        'status': 'Resolved'
    })
    processed_indices.add(i)

```



```

        results.extend(processed_peaks)

    print(f"Deconvolution took {time.time() - start:.2f}s")
    return pd.DataFrame(results)

def calculate_gradient_slope(df, peaks_df, run_groups):
    """Step 7: Calculate dynamic gradient slope at apex using Savitzky-Golay."""
    if 'Conc_B_%' not in df.columns:
        return peaks_df

    final_results = []

    for run_id, group in run_groups:
        run_peaks = peaks_df[peaks_df['run_no'] == run_id].copy()
        if run_peaks.empty:
            continue

        x = group['volume_ml'].values
        y = group['Conc_B_%'].values

        window_len = min(11, len(y) - 1)
        if window_len % 2 == 0:
            window_len -= 1
        if window_len < 4:
            window_len = 3

        try:
            y_smooth = savgol_filter(y, window_len, 3)
            dy_dx = np.gradient(y_smooth, x)
        except:
            dy_dx = np.gradient(y, x)

        for idx, peak in run_peaks.iterrows():
            apex_vol = peak['apex_volume_ml']
            closest_idx = np.searchsorted(x, apex_vol)
            if closest_idx >= len(x):
                closest_idx = len(x) - 1

            slope = dy_dx[closest_idx]
            peak['gradient_slope_at_apex'] = slope
            final_results.append(peak)

    return pd.DataFrame(final_results)

def _ensure_headers_exist(filepath):
    """Helper to check if file exists and has headers."""
    if not os.path.exists(filepath):
        return False
    if os.path.getsize(filepath) == 0:
        return False
    with open(filepath, 'r') as f:
        first_line = f.readline()
        return 'run_no' in first_line

def generate_output(df, peaks_df, run_groups):
    """Generate the markdown output file with append logic."""
    df_dict = {run_id: group.sort_values('volume_ml') for run_id, group in
                run_groups}
    needs_headers = not _ensure_headers_exist(OUTPUT_FILE)

```

```

with open(OUTPUT_FILE, 'a') as f:
    if needs_headers:
        f.write("###_Raw_Detection\n")
        f.write("|_run_no_|_peak_id_|_apex_volume_ml_|_deconvolved_peak_count_|_min_resolution_rs_|_gradient_slope_at_apex_|_sample_code_imputed_|_stage_imputed_|_status_|_n")
        f.write("
|-----|-----|-----|-----|-----|-----|-----|-----|
n")

    for run_id, group in run_groups:
        run_peaks = peaks_df[peaks_df['run_no'] == run_id].copy()
        if run_peaks.empty:
            continue

        run_peaks = run_peaks.sort_values('height', ascending=False)

        total_peaks = len(run_peaks)
        truncation_note = ""
        if total_peaks > MAX_PEAKEs_PER_RUN:
            run_peaks = run_peaks.head(MAX_PEAKEs_PER_RUN)
            truncation_note = f"Summary_truncated:_{total_peaks}_{MAX_PEAKEs_PER_RUN}_additional_peaks_detected"

        run_df = df_dict[run_id]
        run_x = run_df['volume_ml'].values

        for idx, peak in run_peaks.iterrows():
            apex_vol = peak['apex_volume_ml']
            closest_idx = np.searchsorted(run_x, apex_vol)
            if closest_idx >= len(run_x):
                closest_idx = len(run_x) - 1

            row = run_df.iloc[closest_idx]
            sample_code = row['Sample_Code']
            stage = row['chromatography_stage']

            grad_slope = peak.get('gradient_slope_at_apex', np.nan)
            min_rs = peak.get('min_resolution_rs', np.nan)

            apex_str = f"{apex_vol:.2f}" if pd.notna(apex_vol) else ""
            count_str = str(peak['deconvolved_peak_count'])
            rs_str = f"{min_rs:.2f}" if pd.notna(min_rs) else ""
            slope_str = f"{grad_slope:.4f}" if pd.notna(grad_slope) else ""

            f.write(f"|_{run_id}|_{idx+1}|_{apex_str}|_{count_str}|_{rs_str}|_{slope_str}|_{sample_code}|_{stage}|_{peak['status']}|_n")

            if truncation_note:
                f.write(f"|_{run_id}|_NOTE_|_|_|_|_|_|_|_|_|_|_{truncation_note}|_n")

    if needs_headers:
        f.write("\n###_Calculated_Metrics\n")
        f.write("Metrics_calculated_and_included_in_the_table_above.\n")

def main():
    df = load_and_preprocess_data()

```

```

df = clip_negative_signals(df)

# Limit processing to first MAX_RUNS_TO_PROCESS to prevent timeout during
# debugging
unique_runs = df['run_no'].unique()
if len(unique_runs) > MAX_RUNS_TO_PROCESS:
    print(f"Limiting processing to first {MAX_RUNS_TO_PROCESS} runs for debugging.")
    df = df[df['run_no'].isin(unique_runs[:MAX_RUNS_TO_PROCESS])]

df = apply_baseline_correction(df)
df = impute_metadata(df)

run_groups = list(df.groupby('run_no'))

peaks_df = detect_peaks_derivative(df, run_groups)

if not peaks_df.empty:
    peaks_df = deconvolve_peaks(df, peaks_df, run_groups)
    peaks_df = calculate_gradient_slope(df, peaks_df, run_groups)
    generate_output(df, peaks_df, run_groups)
else:
    with open(OUTPUT_FILE, 'a') as f:
        f.write("No peaks detected.\n")

if __name__ == "__main__":
    main()

```

Listing 1: Code.1

```

import pandas as pd
import numpy as np
import os
from scipy.ndimage import maximum_filter1d

# Configuration
INPUT_FILE = "/inputs/chromatography_combined.csv"
OUTPUT_FILE = "/workspace/peak_metrics_raw.md"
WINDOW_SIZE = 50 # Global configuration variable
EPSILON = 1e-10 # Epsilon for numerical stability

# Step 1: Load Data
cols_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
                'fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise'
                ]

try:
    df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)
except FileNotFoundError:
    raise FileNotFoundError(f"Input file not found: {INPUT_FILE}")

# Ensure numeric types
numeric_cols = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
                'fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise'
                ]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors='coerce')

# Helper function to ensure headers exist before writing
def _ensure_headers_exist(filepath):

```

```

if not os.path.exists(filepath):
    return False
if os.path.getsize(filepath) == 0:
    return False
with open(filepath, 'r') as f:
    first_line = f.readline().strip()
    return first_line == "###RawDetection"

def detect_peaks_and_metrics(group):
    """
    Detects peaks using Local Window Thresholding (rolling max of abs derivative).
    Returns a DataFrame of peak metrics.
    """
    group = group.sort_values('volume_ml').reset_index(drop=True)
    uv_280 = group['UV_1_280_mAU'].values
    volume = group['volume_ml'].values

    # Compute absolute derivative
    abs_deriv = np.abs(np.gradient(uv_280))

    # Rolling maximum of absolute derivative within WINDOW_SIZE using scipy (
    # Vectorized)
    window = min(WINDOW_SIZE, len(abs_deriv))
    if window % 2 == 0:
        window += 1
    rolling_max_deriv = maximum_filter1d(abs_deriv, size=window, mode='nearest')

    # Threshold: value > rolling_max_deriv * 0.5
    is_peak = np.zeros(len(uv_280), dtype=bool)

    start_idx = window // 2
    end_idx = len(uv_280) - (window // 2)

    # Vectorized peak detection
    local_max = (uv_280 > np.roll(uv_280, 1)) & (uv_280 > np.roll(uv_280, -1))
    threshold_condition = uv_280 > (rolling_max_deriv * 0.5)

    valid_range = np.zeros(len(uv_280), dtype=bool)
    valid_range[start_idx:end_idx] = True

    is_peak = local_max & threshold_condition & valid_range

    peaks_info = []

    for i, is_p in enumerate(is_peak):
        if is_p:
            peak_idx = i
            peak_height = uv_280[peak_idx]

            # FWHM: 50% height
            half_height = peak_height * 0.5

            # Vectorized search for boundaries
            left_mask = uv_280[:peak_idx+1] <= half_height
            if np.any(left_mask):
                left_idx = np.argmax(left_mask)
            else:
                left_idx = 0

```

```

right_mask = uv_280[peak_idx:] <= half_height
if np.any(right_mask):
    right_idx = peak_idx + np.argmax(right_mask)
else:
    right_idx = len(uv_280) - 1

peak_segment = group.iloc[left_idx:right_idx+1]
if len(peak_segment) < 2:
    continue

v_left = volume[left_idx]
v_right = volume[right_idx]
fwhm = v_right - v_left

height = peak_height

# Area
area_280 = np.trapz(peak_segment['UV_1_280_mAU'].values, peak_segment[
    'volume_ml'].values)
area_260 = np.trapz(peak_segment['UV_2_260_mAU'].values, peak_segment[
    'volume_ml'].values)

if area_280 < EPSILON:
    ratio = np.nan
else:
    ratio = area_260 / (area_280 + EPSILON)

# Tailing Factor (5% height)
five_percent_height = peak_height * 0.05

left_mask_5 = uv_280[:peak_idx+1] <= five_percent_height
l_idx_5 = np.argmax(left_mask_5) if np.any(left_mask_5) else 0

right_mask_5 = uv_280[peak_idx:] <= five_percent_height
r_idx_5 = peak_idx + np.argmax(right_mask_5) if np.any(right_mask_5)
    else len(uv_280) - 1

w_0_05 = volume[r_idx_5] - volume[l_idx_5]
f_dist = volume[peak_idx] - volume[l_idx_5]
tailing_factor = w_0_05 / (2 * f_dist) if f_dist != 0 else np.nan

# Fraction alignment check
frac_min = group.iloc[peak_idx]['fraction_volume_ml_min_precise']
frac_max = group.iloc[peak_idx]['fraction_volume_ml_max_precise']

if pd.isna(frac_min) or pd.isna(frac_max):
    print(f"Run_{group.iloc[0]['run_no']}: Fraction alignment skipped
        for peak at {volume[peak_idx]} due to missing fraction data.")

peaks_info.append({
    'FWHM': fwhm,
    'Height': height,
    'Area_280': area_280,
    'Area_260': area_260,
    'Ratio_260_280': ratio,
    'Tailing_Factor': tailing_factor
})

return pd.DataFrame(peaks_info)

```

```
# Apply peak detection per run
results_list = []
for run_no, group in df.groupby('run_no'):
    peak_df = detect_peaks_and_metrics(group)
    if peak_df.empty:
        results_list.append({'run_no': run_no, 'has_peaks': False})
        continue

    peak_df['run_no'] = run_no
    peak_df = peak_df.sort_values('Height', ascending=False).reset_index(drop=True)
    peak_df['peak_id'] = range(1, len(peak_df) + 1)
    results_list.append(peak_df)

# Prepare output lines
output_lines = []
header_written = _ensure_headers_exist(OUTPUT_FILE)

if not header_written:
    output_lines.append("###RawDetection")
    output_lines.append("Run|PeakID|FWHM|Height|Area_280|Area_260|Ratio_260_280|Tailing_Factor|Tailing_Status|Tol_Lower|Tol_Upper")
    output_lines.append("---|---|---|---|---|---|---|---|---|---|---")
    output_lines.append("\n###CalculatedMetrics")

# Separate runs with peaks and runs without
runs_with_peaks = [r for r in results_list if isinstance(r, pd.DataFrame)]
runs_without_peaks = [r for r in results_list if isinstance(r, dict) and not r['has_peaks']]

# Create set for O(1) lookup
runs_without_peaks_set = {r['run_no'] for r in runs_without_peaks}

if not runs_with_peaks:
    for run_info in runs_without_peaks:
        output_lines.append(f"{run_info['run_no']}|||Nopeaksdetected|-")
else:
    df_peaks = pd.concat(runs_with_peaks, ignore_index=True)

    run_mean_tailing = df_peaks.groupby('run_no')['Tailing_Factor'].mean()
    overall_mean_tailing = run_mean_tailing.mean()
    overall_sd_tailing = run_mean_tailing.std(ddof=1) if len(run_mean_tailing) > 1 else 0.0

    tolerance_lower = overall_mean_tailing - 2 * overall_sd_tailing
    tolerance_upper = overall_mean_tailing + 2 * overall_sd_tailing

    def classify_tailing(tf, tol_lower, tol_upper):
        if pd.isna(tf):
            return "Fail"

        usp_outside = (tf < 0.8 or tf > 1.8)
        tol_outside = (tf < tol_lower or tf > tol_upper)

        if usp_outside and tol_outside:
            return "Outlier"
        elif usp_outside or tol_outside:
```



```

    fraction_volume_ml_max_precise'], errors='coerce')

# 2. Filter and Prepare Data
required_cols = ['run_no', 'volume_ml', 'fraction_volume_ml_min_precise', '
    fraction_volume_ml_max_precise',
    'UV_1_280_mAU', 'UV_2_260_mAU', 'Sample_Code']
df_analysis = df[required_cols].copy()

# Explicit check for missing fraction boundaries before dropping
missing_boundary_count = df_analysis[['fraction_volume_ml_min_precise', '
    fraction_volume_ml_max_precise']].isna().any(axis=1).sum()
if missing_boundary_count > 0:
    print(f"Skipping {missing_boundary_count} rows due to missing fraction
        boundaries")
df_analysis = df_analysis.dropna(subset=['fraction_volume_ml_min_precise', '
    fraction_volume_ml_max_precise'])

# Ensure numeric types
df_analysis['volume_ml'] = pd.to_numeric(df_analysis['volume_ml'], errors='coerce'
    )
df_analysis['UV_1_280_mAU'] = pd.to_numeric(df_analysis['UV_1_280_mAU'], errors='
    coerce')
df_analysis['UV_2_260_mAU'] = pd.to_numeric(df_analysis['UV_2_260_mAU'], errors='
    coerce')
df_analysis = df_analysis.dropna(subset=['volume_ml', 'UV_1_280_mAU', '
    UV_2_260_mAU'])

# 3. Calculate Aggregate 260/280 Area Ratio for Each Fraction
df_analysis['fraction_id'] = df_analysis.groupby(['run_no', '
    fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']).ngroup()

fraction_stats = df_analysis.groupby(['run_no', 'fraction_id']).agg(
    area_280=('UV_1_280_mAU', 'sum'),
    area_260=('UV_2_260_mAU', 'sum'),
    min_vol=('volume_ml', 'min'),
    max_vol=('volume_ml', 'max')
).reset_index()

# Ratio_260_280 is defined as Area Ratio via trapezoidal integration (sum of UV
    values as proxy for area).
# Using EPSILON (1e-9) threshold for area_280 to prevent numerical bias from near-
    zero values.
EPSILON = 1e-9
fraction_stats['ratio'] = np.where(fraction_stats['area_280'] > EPSILON,
    fraction_stats['area_260'] / fraction_stats['area_280'], np.nan)

# 4. Identify Outliers (>2 SD from run mean)
run_stats = fraction_stats.groupby('run_no')['ratio'].agg(['mean', 'std']).
    reset_index()
run_stats.columns = ['run_no', 'ratio_mean', 'ratio_std']

fraction_stats = fraction_stats.merge(run_stats, on='run_no', how='left')

# Handle NaN standard deviations explicitly
fraction_stats['is_outlier'] = False
valid_std_mask = fraction_stats['ratio_std'].notna() & (fraction_stats['ratio_std'
    ] > 0)
outlier_condition = (
    (fraction_stats['ratio'] > (fraction_stats['ratio_mean'] + 2 * fraction_stats[

```



```

        'ratio_std'])) |
        (fraction_stats['ratio'] < (fraction_stats['ratio_mean'] - 2 * fraction_stats[
            'ratio_std']))
    )
fraction_stats.loc[valid_std_mask & outlier_condition, 'is_outlier'] = True

fraction_stats['is_outlier'] = fraction_stats['is_outlier'].fillna(False).astype(
    bool)

# 5. Detect Consecutive Outliers (>= 3 consecutive peaks ordered by volume_ml
    within run_no)
# Ensure data is sorted by volume_ml within each run_no group before applying
    sequence logic
fraction_stats = fraction_stats.sort_values(['run_no', 'min_vol'])

def find_consecutive_sequences(group):
    group = group.copy()
    group['is_outlier'] = group['is_outlier'].astype(bool)
    group['seq_id'] = (group['is_outlier'] != group['is_outlier'].shift()).cumsum(
        ())

    outlier_groups = group[group['is_outlier']].groupby('seq_id')
    valid_seq_ids = outlier_groups.size().loc[lamba x: x >= 3].index

    group['is_consecutive_outlier'] = group['seq_id'].isin(valid_seq_ids)
    return group

fraction_stats = fraction_stats.groupby('run_no', group_keys=False).apply(
    find_consecutive_sequences)

# 6. Generate Visualisation
# Use groupby iterators to avoid creating intermediate dictionaries with potential
    data loss
# Verify that fraction_stats_by_run contains DataFrames with required columns
runs = fraction_stats['run_no'].unique()
n_runs = len(runs)

if n_runs == 0:
    print("No runs found to plot.")
else:
    cols = int(np.ceil(np.sqrt(n_runs)))
    rows = int(np.ceil(n_runs / cols))

    fig, axes = plt.subplots(rows, cols, figsize=(15, 5 * rows))
    if n_runs == 1:
        axes = np.array([axes])
        axes = axes.flatten()

    for i, run in enumerate(runs):
        ax = axes[i]

        # Filter data for the current run directly
        run_data = df_analysis[df_analysis['run_no'] == run].sort_values('
            volume_ml')
        run_fractions = fraction_stats[fraction_stats['run_no'] == run]
        run_consecutive = run_fractions[run_fractions['is_consecutive_outlier']]

        # Verify required columns exist before plotting
        if 'min_vol' not in run_fractions.columns or 'max_vol' not in

```

```

run_fractions.columns:
print(f"Warning: Required columns missing for Run_{run}. Skipping plot
      .")
continue

# Plot UV 280
ax.plot(run_data['volume_ml'], run_data['UV_1_280_mAU'], color='blue',
        linewidth=1, label='UV_280nm')

# Iterative plotting for fraction boundaries to avoid Series ambiguity
if not run_fractions.empty:
    for vol in run_fractions['min_vol'].values:
        ax.axvline(vol, color='gray', linestyle='--', alpha=0.5)
    for vol in run_fractions['max_vol'].values:
        ax.axvline(vol, color='gray', linestyle='--', alpha=0.5)

# Fixed: Iterate over rows to pass scalar values to axvspan instead of
# arrays
if not run_consecutive.empty:
    for idx in run_consecutive.index:
        min_vol = run_consecutive.loc[idx, 'min_vol']
        max_vol = run_consecutive.loc[idx, 'max_vol']
        ax.axvspan(min_vol, max_vol, color='red', alpha=0.3)

ax.set_title(f'Run_{run}: Purity and Consecutive Outlier Analysis')
ax.set_xlabel('volume_ml')
ax.set_ylabel('UV_1_280_mAU')
ax.legend(loc='upper right', fontsize='small')
ax.grid(True, alpha=0.3)

# Remove unused subplots
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.savefig('/workspace/purity_outliers_all_runs.png', dpi=150, bbox_inches='
tight')
plt.close()

```

Listing 3: Code_3